# Importance of memory access efficiency

Floating point operations/s (FLOPS) = floating point ops/cycle × clock cycle

```
07    for (int k = 0; k < Width; ++k) {
08        Pvalue += M[row*Width+k] * N[k*Width+col];
09    }
```

**FIGURE 5.1**

The most executed part of the matrix multiplication kernel in Fig. 3.11.

- In each iteration of the loop, two global memory accesses are performed for one floating-point multiplication & addition
- Floating point operations (FLOPs) per byte access (FLOP/B)

$$= \frac{\text{Floating point operations (FLOP)}}{\text{No. of bytes accessed (B)}}$$

- $\text{FLOP/B} = \frac{2}{8} = 0.25$
- A100 GPU has 1555 GB/s peak global memory bandwidth.
- Since matmul kernel performs 0.25 FLOP/B, the global memory bandwidth limits throughput of single-precision FLOPs:

  1555 GB/s × 0.25 FLOP/B = 389 GFLOPs.

Multiplying 0.25 FLOP/B by 1555 GB/s gives you the **maximum computational throughput a program can achieve when it is completely bottlenecked by memory speed**, which in this case is 389 GigaFLOPs per second (GFLOPS) [1].

Intuitively, here is how the two numbers interact:

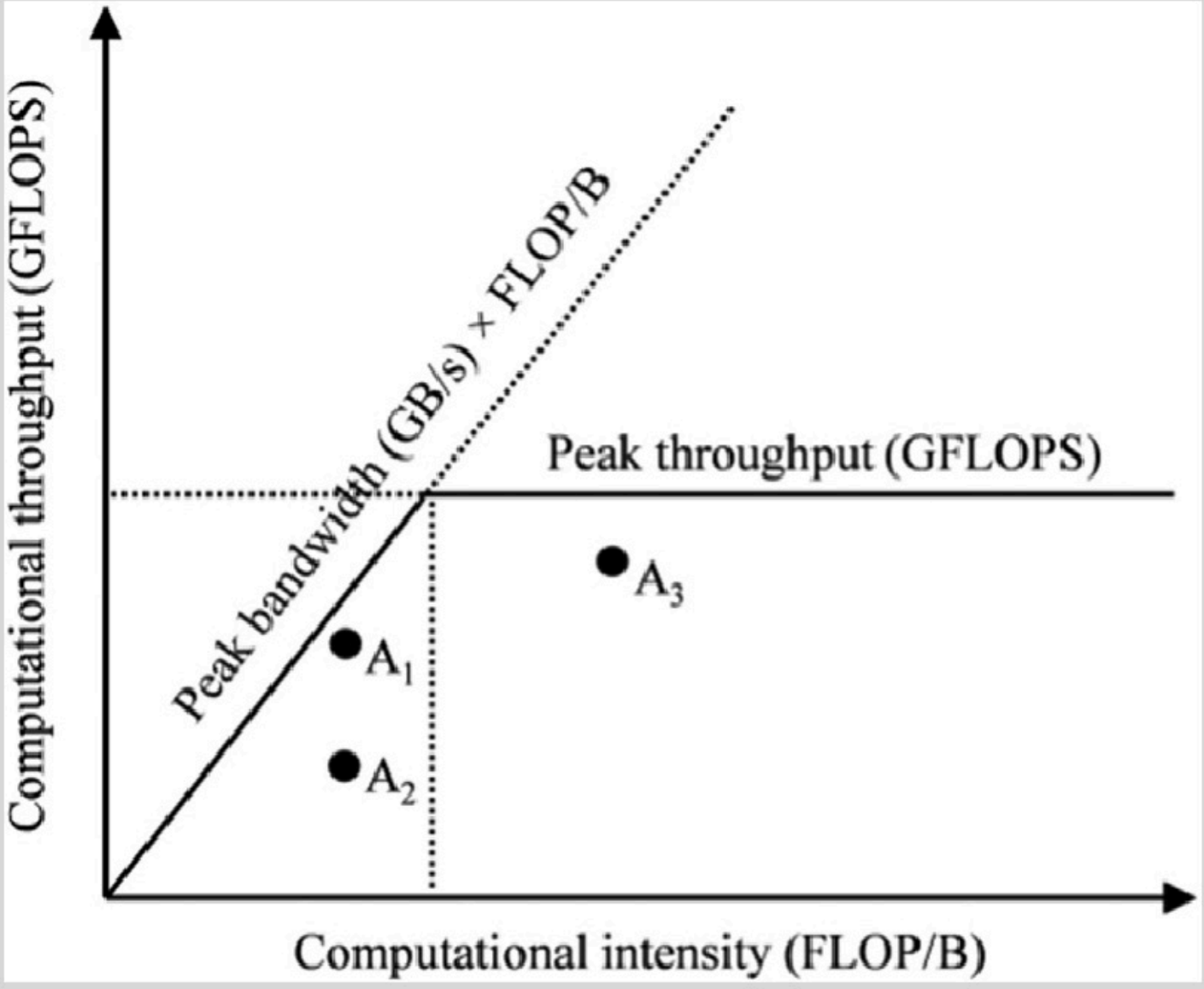
- **0.25 FLOP/B** is the program's **compute to global memory access ratio** (also called

computational intensity) [2]. It means that for every 1 byte of data your program pulls from memory, it only performs 0.25 floating-point operations [2].

• **1555 GB/s** is the **peak global memory bandwidth** of the hardware (specifically the Ampere A100 GPU) [1]. This is the absolute fastest rate at which the hardware can deliver data from memory to the processing cores [1] [3].

When you multiply the rate of data delivery (1555 GB/s) by the amount of work done per byte of data (0.25 FLOP/B), the "bytes" cancel out. The result—**389 GFLOPS**—tells you exactly how fast your program will run based purely on how quickly it can be fed data [1].

Roofline Model




Because A1 and A2 have the exact same computational intensity, they are performing the same amount of mathematical work for every byte of data loaded. What differentiates a point lower on the y-axis is that **it is failing to pull those bytes from memory at the hardware's maximum possible speed**

For a highly unoptimized algorithm like A2, you can improve its throughput by applying optimizations that improve memory bandwidth utilization, effectively moving it straight up the y-axis closer to A1. However, once an application is efficiently utilizing the memory bandwidth (like A1), the only way to achieve higher computational throughput is to change the algorithm to **increase its computational intensity (FLOP/B)**, which moves the point to the right along the x-axis so it can climb higher up the sloped roofline

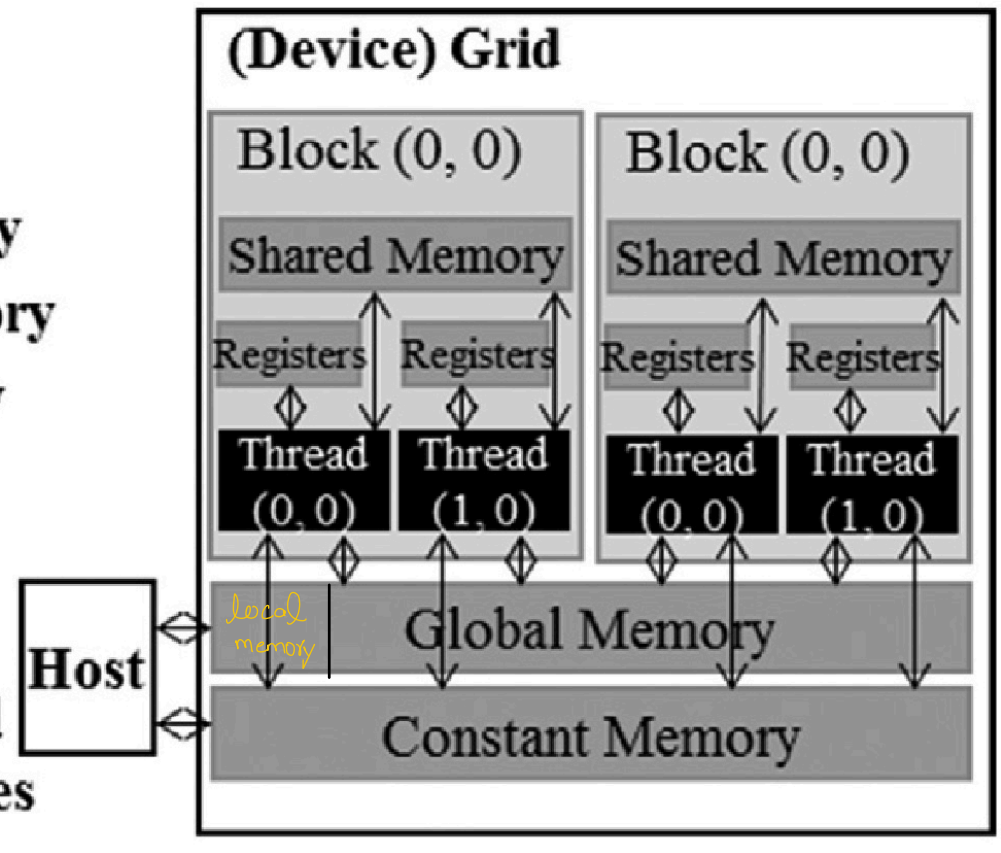
Cuda memory types

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

Host code can

- Transfer data to/from per grid **global** and **constant memories**




The quote you highlighted provides an exception to this rule: **CUDA memory fencing** [4]. Fencing is a technique that forces the GPU to guarantee that any data a block has written to global memory is fully updated, coherent, and visible to other blocks before the program is allowed to continue executing.

• **Streaming Multiprocessors (SMs)** are the physical hardware processing units on the GPU that execute your thread blocks.

• If you launch a kernel where the **number of blocks is smaller than the number of SMs**, the GPU has enough physical hardware to schedule and run *every single block* at the exact same time. Because all blocks are "co-resident" (actively running on the hardware simultaneously), one block can safely use a memory fence to wait for a signal or data from another block, knowing that the other block is actively computing and will eventually send the data.

• If you launch **more blocks than SMs**, the GPU does not have enough hardware to run them all at once. It must place the extra blocks into a queue, waiting for the currently active blocks to finish and free up the SMs. If an active block uses a memory fence to wait for data that is supposed to be written by a block that is still stuck in the queue, your program will **deadlock** (freeze indefinitely). The active block will never finish because it is waiting for the queued block, and the queued block can never start because the active block refuses to give up the SM.

Therefore, using memory fencing to synchronize across blocks is only safe if you constrain the grid size to ensure all blocks are guaranteed to be running on the hardware at the exact same time [4].

In CUDA, `__threadfence()` is used to solve a specific problem with how memory writes are ordered and made visible across different thread blocks.

Normally, **there is no guarantee that if one block writes data to global memory, other blocks will "see" it**, nor is there a guarantee on the order in which those writes become visible to the rest of the grid [1].

For example, imagine a scenario where Block A computes some data, writes it to global memory, and then uses an atomic operation to set a "ready flag" for Block B. Without memory fencing, it is entirely possible that Block B sees the "ready flag" but still reads incorrect or incomplete data because Block A's original data

writes haven't fully settled in memory yet [1].

The `__threadfence()` function fixes this by **ensuring the strict ordering of memory writes** [2]. It enforces a rule that all memory writes placed *before* the fence must be fully visible to all other blocks *before* any memory writes placed *after* the fence [2]. Importantly, it does this efficiently; it doesn't necessarily stall the current thread until everything is completely visible across the entire grid, which would otherwise severely hurt performance [2].

Here is the exact step-by-step example of how it is used to safely pass data:

1. First, you **store your data** to global memory [2].

2. Next, you call **`__threadfence()`** [2].

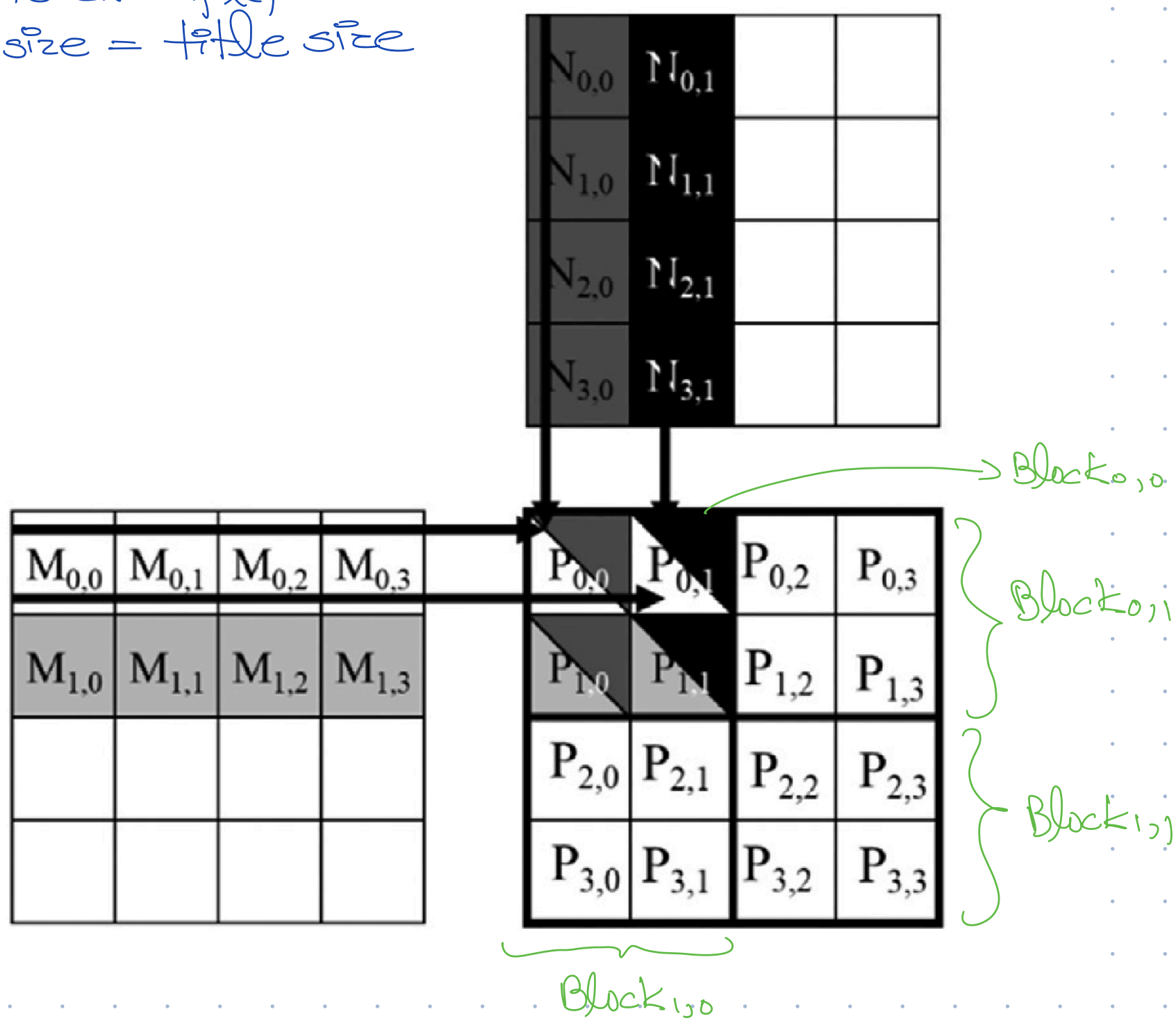
3. Finally, you **atomically mark a flag** indicating that the data is ready [3].

Because the fence enforces the order of operations, you now have a strict guarantee: **if another block sees that the flag is marked, it will definitely also see the complete and correct data** [3].

This directly explains the textbook line from our earlier conversation—`__threadfence()` is the mechanism you use to ensure data coherence when thread blocks need to collaborate during a single kernel's execution!

# Tiling for reduced memory traffic

For this example, our block size = tile size




- We use four 2x2 blocks to compute the P matrix
- The four threads in block0,0 compute P0,0 P0,1 P1,0 P1,1

Access order

| | | | | |
|---|---|---|---|---|
| $thread_{0,0}$ | $M_{0,0} * N_{0,0}$ | $M_{0,1} * N_{1,0}$ | $M_{0,2} * N_{2,0}$ | $M_{0,3} * N_{3,0}$ |
| $thread_{0,1}$ | $M_{0,0} * N_{0,1}$ | $M_{0,1} * N_{1,1}$ | $M_{0,2} * N_{2,1}$ | $M_{0,3} * N_{3,1}$ |
| $thread_{1,0}$ | $M_{1,0} * N_{0,0}$ | $M_{1,1} * N_{1,0}$ | $M_{1,2} * N_{2,0}$ | $M_{1,3} * N_{3,0}$ |
| $thread_{1,1}$ | $M_{1,0} * N_{0,1}$ | $M_{1,1} * N_{1,1}$ | $M_{1,2} * N_{2,1}$ | $M_{1,3} * N_{3,1}$ |

- The above table shows all the memory accesses done by all the threads in block0,0
- From the table, we can there is significant overlap in M & N elements accessed by the four threads.
- For example, $thread_{0,0}$ & $thread_{0,1}$ both access the complete row of M.

  Similarly, $thread_{0,1}$ & $thread_{1,1}$ both access the complete column 1 of N.
- If we make it so that $thread_{0,0}$ & $thread_{0,1}$ collaborate so that these M elements are loaded once only, then we will reduce memory access by half

```
01    #define TILE_WIDTH 16
02    __global__ void matrixMulKernel(float* M, float* N, float* P, int Width) {
03
04        __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
05        __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
06
07        int bx = blockIdx.x;  int by = blockIdx.y;
08        int tx = threadIdx.x; int ty = threadIdx.y;
09
10        // Identify the row and column of the P element to work on
11        int Row = by * TILE_WIDTH + ty;
12        int Col = bx * TILE_WIDTH + tx;
13
14        // Loop over the M and N tiles required to compute P element
15        float Pvalue = 0;
16        for (int ph = 0; ph < Width/TILE_WIDTH; ++ph) {
17
18            // Collaborative loading of M and N tiles into shared memory
19            Mds[ty][tx] = M[Row*Width + ph*TILE_WIDTH + tx];
20            Nds[ty][tx] = N[(ph*TILE_WIDTH + ty)*Width + Col];
```

```
21            __syncthreads();
22
23            for (int k = 0; k < TILE_WIDTH; ++k) {
24                Pvalue += Mds[ty][k] * Nds[k][tx];
25            }
26            __syncthreads();
27
28        }
29        P[Row*Width + Col] = Pvalue;
30
31    }
```

Remember that each block is assigned a specific tile of the output matrix *P* to calculate. To compute the final values for that specific output tile, the block must perform a complete dot product by stepping through all the necessary pairs of *M* and *N* tiles. The outer loop (which manages the phases) starts at ph = 0 for *every* block in the grid

Do title width need to same as block size?

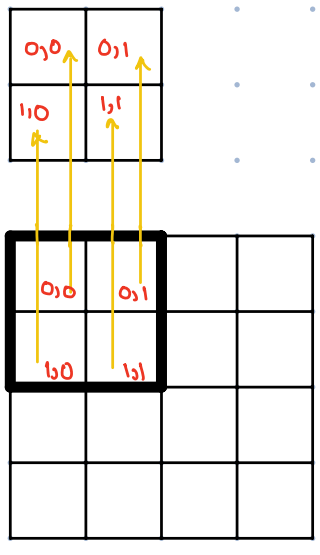
**Why you might choose (or not choose) to make them different:** The fundamental rule of tiling is simply that the chosen tile size must be small enough to fit into the limited capacity of the on-chip shared memory [1]. If you were to make the block size and tile size unequal, you would lose that simple 1-to-1 mapping:

• **If the tile is larger than the thread block:** You would not have enough threads to load the whole tile at once. Your code would become more complex because each individual thread would have to execute a loop to load multiple elements into shared memory per phase.

• **If the tile is smaller than the thread block:** You would have more threads than data elements. During the memory loading phase, some threads would sit idle doing nothing, which is an inefficient use of the GPU's resources.

By keeping the block size and tile size equal, you guarantee that all threads are utilized evenly during the memory loading phase without requiring complex looping logic [2] [3].

Q10.




Row = blockIdx.y * Bz + threadIdx.y

col = blockIdx.x * Bz + threadIdx.x

baseIdx = Row * A_width + col

1. For Blocksize = 1, the kernel will run correctly.

Mathematically, the transpose of a 1×1 matrix (a single scalar value) is simply the value itself. By reading the element and writing it back to the identical location, the kernel successfully "transposes" the 1×1 block by doing absolutely nothing to it.

2. We need __syncthread() after line 10.

Q2.

$$\frac{\text{flops}}{\text{Byte}}$$

$$\frac{36}{7 \times 32/8} = 1.28$$

a.

$= 1.28 \text{ Flop/B} \times 100 \text{ GB/s}$

$= 128.57 \text{ GFlop/s} << 200 \text{ GFlop/s}$

So, memory-bound

b.

$= 1.28 \times 250$

$= 321 >> 300$

So, compute-bound

Q11.

e. $4 + 128 \times 4 = 516$ bytes

f.

```
01  __global__ void foo_kernel(float* a, float* b) {
02     unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
03     float x[4];
04     __shared__ float y_s;
05     __shared__ float b_s[128];
06     for(unsigned int j = 0; j < 4; ++j) {
07        x[j] = a[j*blockDim.x*gridDim.x + i];
08     }
09     if(threadIdx.x == 0) {
10        y_s = 7.4f;
11     }
12     b_s[threadIdx.x] = b[i];
13     __syncthreads();
14     b[i] = 2.5f*x[0] + 3.7f*x[1] + 6.3f*x[2] + 8.5f*x[3]
15           + y_s*b_s[threadIdx.x] + b_s[(threadIdx.x + 3)%128];
16  }
17  void foo(int* a_d, int* b_d) {
18     unsigned int N = 1024;
19     foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d);
20  }
```

$$= \frac{10 \text{ flops}}{1 \times 4 + 4 \times 4 \text{ global accesses}}$$

$$= 0.50 \text{ OB/B}$$

Q12.

a. $\frac{2048}{64} = 32$ blocks ✓

$2048 \times 27 = 55\text{k}$ registers ✓

$$\frac{4096 \text{ Byte shared memory}}{64} = 64 \text{ Byte/thread}$$

$$\frac{96 \text{ kB}}{64 \text{ B/thread}} = 1536 \text{ threads}$$

$$\frac{1536}{2048} = 0.75$$

b. $\frac{2048}{256} = 8$ blocks ✓

$2048 \times 31 = 63,488$ registers ✓

$\frac{8 \times 1024}{256} = 32\ B/thread$

$\frac{96 \times 1024\ B}{32\ B/thread} = 3072$

So, Launch 2048 threads, which gives us 100 occupancy.